# MySQL User-Defined Functions (Stored Functions): Complete Notes from Beginner to Advanced

**Level:** Beginner → Intermediate → Advanced
**Database:** MySQL 8.0 / MySQL 8.4
**Language:** SQL
**Purpose:** Learn the theory, syntax, implementation, practical usage, limitations, and optimization of MySQL stored functions.

---

## Table of Contents


1. Introduction to Functions in MySQL
2. What Is a User-Defined Function?
3. Why Do We Need Functions?
4. Types of Functions in MySQL
5. Stored Function vs Stored Procedure
6. Basic Syntax of `CREATE FUNCTION`
7. Creating Your First Function
8. Parameters and Return Types
9. The `RETURN` Statement
10. Local Variables and `DECLARE`
11. Using `SET` and `SELECT ... INTO`
12. Conditional Statements: `IF`, `ELSEIF`, `ELSE`
13. The `CASE` Statement
14. Loops in MySQL Functions
15. Functions with Mathematical Operations
16. Functions with Strings
17. Functions with Dates and Times
18. Functions That Read Table Data
19. Functions with `NULL` Values
20. Error Handling and Validation

21. Deterministic and Non-Deterministic Functions
22. SQL Data Access Characteristics
23. Security and Permissions
24. Restrictions of Stored Functions
25. Advanced Examples and Real-World Use Cases
26. Function Management Commands
27. Debugging and Common Errors
28. Performance Optimization
29. Best Practices
30. Interview Questions and Answers
31. Practice Exercises
32. Quick Revision Cheat Sheet
33. Official References

---

# 1. Introduction to Functions in MySQL

A function is a reusable piece of logic that accepts input, performs an operation, and returns a result.

Functions are widely used in programming languages and databases to simplify repetitive operations.

For example, consider the following calculation:

```
SELECT 1000 * 0.18;
```

This calculates 18% of 1000.

If the same calculation is required in multiple queries, you might repeatedly write the same expression.

Instead, you can create a function:

```
SELECT calculate_tax(1000);
```

The function can perform the calculation internally and return the result.

## 1.1 What does a function do?

A typical function follows three steps:

1. Accept input parameters.
2. Execute the required logic.
3. Return a value.

Conceptually:

```
Input → Processing → Output
```

Example:

```
Input: 1000
Operation: Calculate 18% tax
Output: 180
```

## 1.2 Functions in everyday applications

Functions can perform many types of operations:

- Calculate discounts, taxes, commissions, and salaries.
- Convert temperatures or currencies using supplied rates.
- Format names and other text.
- Calculate a person's age from a birth date.
- Determine whether a number is even or odd.
- Retrieve a particular value from a database table.
- Classify orders based on their amounts.
- Calculate business-specific metrics.

**Important:** A stored function returns a single scalar value, such as a number, string, date, or other supported SQL value. It is not designed to return a result set containing multiple rows and columns.

---

# 2. What Is a User-Defined Function?

A user-defined function is a function created by a developer to implement custom logic that is not already available as a suitable built-in function.

In MySQL, there is an important terminology distinction.

### A. Stored function

A stored function is written in SQL and saved inside the database.

Example:

```
CREATE FUNCTION square_number(n INT)
RETURNS INT
DETERMINISTIC
RETURN n * n;
```

Usage:

```
SELECT square_number(5);
```

Output:

```
25
```

This is the main subject of these notes.

### B. Loadable UDF

In strict MySQL terminology, a UDF traditionally refers to a loadable function implemented in compiled code, such as C or C++, and installed into the MySQL server.

Such functions extend MySQL beyond ordinary SQL routines and require additional server-level deployment and security considerations.

For most SQL learning, database development, and interviews, the term *user-defined function* commonly refers to a stored function.

Reference: https://dev.mysql.com/doc/refman/8.4/en/server-loadable-functions.html

## 2.1 Stored functions versus built-in functions

MySQL already provides built-in functions.

Examples:

```
SELECT UPPER('mysql');
SELECT LOWER('MYSQL');
SELECT LENGTH('Database');
SELECT ABS(-25);
SELECT ROUND(12.567, 2);
```

Expected results:

```
MYSQL
mysql
8
25
12.57
```

You create a stored function when your application needs custom reusable behavior.

For example:

```
SELECT calculate_bonus(50000, 10);
```

This could return a 10% bonus on a salary of 50,000.

---

# 3. Why Do We Need Functions?

Functions help organize database logic and make frequently used calculations reusable.

## 3.1 Code reusability

Suppose several reports calculate a 10% commission.

Without a function:

```
SELECT employee_name, salary * 0.10 AS commission
FROM employees;
```

Another query might repeat the same calculation:

```
SELECT department, SUM(salary * 0.10) AS total_commission
FROM employees
GROUP BY department;
```

A function can centralize the business rule:

```
SELECT employee_name, calculate_commission(salary)
FROM employees;
```

The calculation is defined once and can be reused.

## 3.2 Better maintainability

Suppose the commission rate changes from 10% to 12%.

If the rule is duplicated across many queries, every copy may need to be changed.

If all queries use the same function, the rule can be updated centrally.

### 3.3 Consistency

A shared function helps ensure that different reports apply the same calculation rules.

For example, all departments can use the same bonus calculation rather than implementing slightly different formulas.

### 3.4 Simpler queries

A complicated expression:

```
SELECT
    employee_name,
    salary * commission_rate / 100 AS commission
FROM employees;
```

Can become:

```
SELECT
    employee_name,
    calculate_commission(salary, commission_rate) AS commission
FROM employees;
```

The second query communicates the business purpose more clearly.

### 3.5 Important limitation

A function is not automatically faster than an ordinary SQL expression.

Calling a stored function once for each row in a large result set can increase execution overhead. Functions that query other tables may be especially expensive.

Use functions for clear, reusable logic, but measure performance for large workloads.

---

## 4. Types of Functions in MySQL

MySQL functions can be classified into several categories.

### 4.1 Built-in functions

These are provided by MySQL.

Examples:

| Category | Functions | Purpose |
|---|---|---|
| Numeric | `ABS()`, `ROUND()`, `CEIL()` | Mathematical operations |
| String | `CONCAT()`, `UPPER()`, `SUBSTRING()` | Text processing |
| Date and time | `NOW()`, `CURDATE()`, `DATEDIFF()` | Date calculations |
| Aggregate | `COUNT()`, `SUM()`, `AVG()` | Calculations across rows |
| Conditional | `IF()`, `COALESCE()`, `NULLIF()` | Conditional expressions |
| Conversion | `CAST()`, `CONVERT()` | Type conversion |

## 4.2 Stored functions

These are custom SQL functions created with `CREATE FUNCTION`.

Example:

```
CREATE FUNCTION add_numbers(a INT, b INT)
RETURNS INT
DETERMINISTIC
RETURN a + b;
```

## 4.3 Loadable functions

These extend the MySQL server using compiled libraries or supported components.

They are a more advanced server-extension topic and are different from SQL stored functions.

## 4.4 Stored functions versus aggregate functions

An aggregate function calculates a value from a collection of rows.

Example:

```
SELECT SUM(salary)
FROM employees;
```

A stored function typically accepts explicit parameters and returns one value.

Example:

```
SELECT add_numbers(10, 20);
```

A stored function may itself use aggregate functions internally when permitted, for example by selecting `SUM()` from a table into a local variable.

---

# 5. Stored Function vs Stored Procedure

Both functions and procedures are stored routines, but they have different purposes.

| Feature | Stored Function | Stored Procedure |
|---|---|---|
| Creation | `CREATE FUNCTION` | `CREATE PROCEDURE` |
| Execution | Used in an expression | Called using `CALL` |
| Return value | Must return one value | No mandatory scalar return |
| Parameters | Input parameters only | `IN`, `OUT`, `INOUT` |
| Use in `SELECT` | Yes | Not directly as a scalar expression |
| Return a result set | No | Yes |
| Explicit transaction control | Not permitted | Permitted subject to applicable rules |
| Typical use | Calculations and scalar transformations | Multi-step operations and workflows |

## 5.1 Example of a function

```
DELIMITER //

CREATE FUNCTION multiply_numbers(a INT, b INT)
RETURNS INT
DETERMINISTIC
BEGIN
    RETURN a * b;
END //

DELIMITER ;
```

Execute it:

```
SELECT multiply_numbers(4, 5);
```

Output:

```
20
```

## 5.2 Example of a procedure

```
DELIMITER //

CREATE PROCEDURE show_message()
BEGIN
    SELECT 'Hello from MySQL' AS message;
END //

DELIMITER ;
```

Execute it:

```
CALL show_message();
```

The procedure returns a result set to the client.

## 5.3 When should you choose a function?

Choose a function when the primary requirement is to calculate and return one value.

Examples:

- Calculate an annual bonus.
- Return a product's discounted price.
- Format a person's full name.
- Determine a grade from marks.
- Calculate the number of days between two dates.

Choose a procedure when the operation involves a workflow, multiple result sets, output parameters, or transaction management.

For example, a monthly payroll process that updates multiple tables is generally better implemented as a procedure or application workflow.

---

# 6. Basic Syntax of `CREATE FUNCTION`

The general syntax is:

```
CREATE FUNCTION function_name (
    parameter_name data_type,
    parameter_name data_type
)
RETURNS return_data_type
[characteristics]
BEGIN
    -- Declare local variables here
    -- Execute statements here
    RETURN expression;
END;
```

The square brackets indicate optional syntax elements, not characters to type into SQL.

## 6.1 Explanation of each component

**`CREATE FUNCTION`**

Creates a stored function in the database.

**`function_name`**

The name used to call the function later.

Example:

```
calculate_bonus
```

### Parameter list

Defines the values supplied by the caller.

Example:

```
salary DECIMAL(10,2),
percentage DECIMAL(5,2)
```

Function parameters are input parameters. Unlike procedure parameters, stored function parameters do not use `OUT` or `INOUT`.

**`RETURNS`**

Declares the data type of the value returned by the function.

Example:

```
RETURNS DECIMAL(10,2)
```

### Characteristics

These describe the function's behavior, such as whether it is deterministic and whether it reads or modifies SQL data.

Example:

```
DETERMINISTIC
READS SQL DATA
```

The correct characteristics depend on what the function actually does.

### `BEGIN` and `END`

Define a compound statement containing multiple SQL statements.

They are not required for every function. A function whose body consists of a single `RETURN` statement can be shorter.

### `RETURN`

Returns the function's scalar result.

A stored function must have a valid way to return a value on every execution path.

---

# 7. Creating Your First Function

Let's create a simple function that adds two numbers.

## 7.1 Function definition

```
DELIMITER //

CREATE FUNCTION add_numbers(
    a INT,
    b INT
)
RETURNS INT
DETERMINISTIC
BEGIN
    RETURN a + b;
END //

DELIMITER ;
```

## 7.2 Understanding `DELIMITER`

Normally, the MySQL client uses a semicolon to identify the end of a statement.

However, the function body itself contains SQL statements terminated by semicolons.

If the client treated the first semicolon inside the function as the end of the complete `CREATE FUNCTION` statement, the definition would be cut off.

Therefore, in clients that support the `DELIMITER` command, we temporarily change the statement terminator.

```
DELIMITER //
```

Now the complete function definition ends with:

```
END //
```

After creation:

```
DELIMITER ;
```

This restores the normal delimiter.

**Important:** `DELIMITER` is a client command, not an SQL statement sent to the MySQL server. Not every database GUI or API accepts it. In those environments, use the client's stored-routine editor or its supported execution method.

## 7.3 Calling the function

```
SELECT add_numbers(10, 20) AS result;
```

Output:

```
+--------+
| result |
+--------+
|     30 |
+--------+
```

## 7.4 Calling a function with table data

Suppose a table contains employee salaries:

```
CREATE TABLE employees (
    employee_id INT PRIMARY KEY,
    employee_name VARCHAR(100),
    salary DECIMAL(10,2)
);
```

Insert sample records:

```
INSERT INTO employees
VALUES
(1, 'Asha', 30000),
(2, 'Rahul', 45000),
(3, 'Neha', 60000);
```

Now use the function with a column expression:

```
SELECT
    employee_name,
    salary,
    add_numbers(salary, 1000) AS salary_plus_1000
FROM employees;
```

The function executes for each row that requires evaluation.

Expected results:

| employee_name | salary | salary_plus_1000 |
|---|---|---|
| Asha | 30000.00 | 31000.00 |
| Rahul | 45000.00 | 46000.00 |
| Neha | 60000.00 | 61000.00 |

---

# 8. Parameters and Return Types

## 8.1 What is a parameter?

A parameter is a named input value that a function receives when it is called.

Example:

```
CREATE FUNCTION square_number(n INT)
RETURNS INT
DETERMINISTIC
RETURN n * n;
```

Here, `n` is the parameter.

When you execute:

```
SELECT square_number(6);
```

The value `6` is supplied to `n`.

The function calculates:

```
6 × 6 = 36
```

## 8.2 Multiple parameters

```
DELIMITER //

CREATE FUNCTION calculate_area(
    length_value DECIMAL(10,2),
    width_value DECIMAL(10,2)
)
RETURNS DECIMAL(20,4)
DETERMINISTIC
BEGIN
    RETURN length_value * width_value;
END //

DELIMITER ;
```

Usage:

```
SELECT calculate_area(10, 5);
```

Output:

```
50.0000
```

## 8.3 Return types

Common return types include:

| Return type | Typical purpose |
|---|---|
| `INT` | Whole-number results |
| `BIGINT` | Large integer results |
| `DECIMAL(p,s)` | Exact decimal calculations |
| `DOUBLE` | Approximate floating-point calculations |
| `VARCHAR(n)` | Text |
| `CHAR(n)` | Fixed-length text |
| `DATE` | Calendar date |
| `DATETIME` | Date and time |
| `TIME` | Time or duration |
| `BOOLEAN` / `BOOL` | Boolean-like values represented as `TINYINT(1)` in MySQL |

For financial calculations, `DECIMAL` is usually preferable to floating-point types because it supports exact decimal arithmetic within its declared precision.

## 8.4 Choosing precision and scale

Consider:

```
RETURNS DECIMAL(10,2)
```

Here:

- `10` is the maximum total number of decimal digits.
- `2` is the number of digits after the decimal point.
- The remaining digits are available before the decimal point.

For example:

```
12345678.90
```

contains eight digits before the decimal point and two after it.

Choose precision and scale according to the expected range and the rounding requirements of your application.

### 8.5 A function with no parameters

A function can have an empty parameter list.

```
DELIMITER //

CREATE FUNCTION get_greeting()
RETURNS VARCHAR(50)
DETERMINISTIC
RETURN 'Welcome to MySQL';

DELIMITER ;
```

Call it:

```
SELECT get_greeting();
```

For a function that always returns the same literal, `DETERMINISTIC` is appropriate.

---

## 9. The `RETURN` Statement

The `RETURN` statement supplies the value that the function gives back to the caller.

Example:

```
DELIMITER //

CREATE FUNCTION double_number(n INT)
RETURNS INT
DETERMINISTIC
BEGIN
    RETURN n * 2;
END //

DELIMITER ;
```

Usage:

```
SELECT double_number(8);
```

Output:

```
16
```

## 9.1 Returning an expression

You can return calculations directly.

```
RETURN a + b;
```

```
RETURN price * (1 - discount_rate / 100);
```

```
RETURN UPPER(customer_name);
```

```
RETURN DATEDIFF(end_date, start_date);
```

## 9.2 Returning a value based on a condition

```
DELIMITER //

CREATE FUNCTION check_number(n INT)
RETURNS VARCHAR(20)
DETERMINISTIC
BEGIN
    IF n > 0 THEN
        RETURN 'Positive';
    ELSEIF n < 0 THEN
        RETURN 'Negative';
    ELSE
        RETURN 'Zero';
    END IF;
END //

DELIMITER ;
```

Usage:

```
SELECT
    check_number(10),
    check_number(-5),
    check_number(0);
```

Output:

```
Positive | Negative | Zero
```

### 9.3 `RETURN` versus `SELECT`

These statements serve different purposes.

```
RETURN result_value;
```

Returns the scalar result of a stored function.

By contrast:

```
SELECT result_value;
```

Normally produces a result set.

A stored function cannot directly return a result set to its caller. When a query is needed inside a function, use `SELECT ... INTO` to assign a scalar result to a local variable.

For example:

```
SELECT COUNT(*)
INTO total_count
FROM employees;
```

Then:

```
RETURN total_count;
```

---

## 10. Local Variables and `DECLARE`

Local variables store temporary values while the function executes.

They are useful when the calculation requires multiple steps.

### 10.1 Syntax

```
DECLARE variable_name data_type DEFAULT value;
```

Example:

```
DECLARE total_amount DECIMAL(10,2) DEFAULT 0;
```

A local variable exists within the stored program's scope. It is not a permanent table column or a session variable.

## 10.2 Example: Calculate a bonus

```
DELIMITER //

CREATE FUNCTION calculate_bonus(
    salary DECIMAL(10,2)
)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE bonus DECIMAL(10,2) DEFAULT 0;

    SET bonus = salary * 0.10;

    RETURN bonus;
END //

DELIMITER ;
```

Usage:

```
SELECT calculate_bonus(50000);
```

Output:

```
5000.00
```

## 10.3 Why declare a variable?

A variable is useful when:

- A calculation requires several intermediate values.
- A query result needs to be stored before further processing.
- The result depends on multiple conditions.
- You need a counter for a loop.
- You want to make complex logic easier to read.

## 10.4 Declaration order

In a compound block, declarations must appear before executable statements.

The general order is:

1. Local variables and conditions.

2. Cursors, if needed.
3. Handlers.
4. Executable statements.

For example:

```
BEGIN
    DECLARE subtotal DECIMAL(10,2) DEFAULT 0;
    DECLARE tax_amount DECIMAL(10,2) DEFAULT 0;

    SET subtotal = 100;
    SET tax_amount = subtotal * 0.18;

    RETURN subtotal + tax_amount;
END
```

The example illustrates the declaration and execution order; place it inside a complete function definition when running it.

---

# 11. Using `SET` and `SELECT ... INTO`

MySQL provides several ways to assign values to local variables.

## 11.1 Assignment using `SET`

```
SET total = 100;
```

With an expression:

```
SET total = price * quantity;
```

With a function:

```
SET full_name = CONCAT(first_name, ' ', last_name);
```

## 11.2 Assignment using `SELECT ... INTO`

Use this form when you need to obtain a value from a query.

Suppose a table contains product prices:

```
CREATE TABLE products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100),
    price DECIMAL(10,2)
);
```

Example function:

```
DELIMITER //

CREATE FUNCTION get_product_price(
    p_product_id INT
)
RETURNS DECIMAL(10,2)
READS SQL DATA
BEGIN
    DECLARE p_price DECIMAL(10,2) DEFAULT NULL;

    SELECT price
    INTO p_price
    FROM products
    WHERE product_id = p_product_id;

    RETURN p_price;
END //

DELIMITER ;
```

Usage:

```
SELECT get_product_price(1);
```

This function reads a price from the `products` table.

**Important:** The example assumes that the ID exists. If no matching row exists, `SELECT ... INTO` raises a `NOT FOUND` condition. The default value of `p_price` does not, by itself, prevent that condition from occurring. Error handling is discussed later.

## 11.3 Difference between `SET` and `SELECT ... INTO`

| `SET` | `SELECT ... INTO` |
|---|---|
| Assigns an expression to a variable | Assigns query result columns to variables |
| Does not require a table | Usually used to retrieve values from a query |

| `SET` | `SELECT ... INTO` |
|---|---|
| Suitable for arithmetic and expressions | Suitable for single-row query results |
| Example: `SET x = a + b` | Example: `SELECT price INTO x FROM products WHERE product_id = 1` |

A `SELECT ... INTO` statement should return no more than one row when assigning to a scalar variable list. Multiple matching rows cause an error.

---

# 12. Conditional Statements: `IF`, `ELSEIF`, `ELSE`

Conditional statements allow a function to choose which logic to execute.

## 12.1 Basic syntax

```
IF condition THEN
    statements;
ELSEIF another_condition THEN
    statements;
ELSE
    statements;
END IF;
```

## 12.2 Example: Pass or fail

```
DELIMITER //

CREATE FUNCTION get_result(
    marks INT
)
RETURNS VARCHAR(10)
DETERMINISTIC
BEGIN
    IF marks >= 40 THEN
        RETURN 'Pass';
    ELSE
        RETURN 'Fail';
    END IF;
END //

DELIMITER ;
```

Usage:

```
SELECT get_result(75);
SELECT get_result(25);
```

Results:

```
Pass
Fail
```

This example assumes the input is already a valid mark. Production code should also validate the permitted range.

## 12.3 Example: Grade calculation

Rules:

- Marks greater than or equal to 90: A
- Marks greater than or equal to 75: B
- Marks greater than or equal to 60: C
- Marks greater than or equal to 40: D
- Otherwise: F

```
DELIMITER //

CREATE FUNCTION calculate_grade(
    marks INT
)
RETURNS CHAR(1)
DETERMINISTIC
BEGIN
    IF marks IS NULL OR marks < 0 OR marks > 100 THEN
        RETURN NULL;
    ELSEIF marks >= 90 THEN
        RETURN 'A';
    ELSEIF marks >= 75 THEN
        RETURN 'B';
    ELSEIF marks >= 60 THEN
        RETURN 'C';
    ELSEIF marks >= 40 THEN
        RETURN 'D';
    ELSE
        RETURN 'F';
    END IF;
END //

DELIMITER ;
```

Usage:

```
SELECT
    calculate_grade(95) AS grade_95,
    calculate_grade(80) AS grade_80,
    calculate_grade(65) AS grade_65,
    calculate_grade(30) AS grade_30;
```

Output:

```
A | B | C | F
```

Returning `NULL` for invalid input is a design choice, not the only possible policy. An application might instead require a clear SQL error.

## 12.4 Why does the order of conditions matter?

Conditions are evaluated in order.

If you test:

```
IF marks >= 40 THEN
    RETURN 'D';
ELSEIF marks >= 90 THEN
    RETURN 'A';
END IF;
```

A score of 95 matches the first condition, so the function returns `D`.

Always put more specific or higher-threshold conditions first when they overlap.

---

# 13. The `CASE` Statement

The `CASE` statement provides another way to implement conditional logic.

## 13.1 Simple `CASE`

```
CASE status_code
    WHEN 1 THEN
        RETURN 'Pending';
    WHEN 2 THEN
        RETURN 'Approved';
    WHEN 3 THEN
        RETURN 'Rejected';
    ELSE
        RETURN 'Unknown';
END CASE;
```

This compares one expression against multiple values.

## 13.2 Searched `CASE`

```
CASE
    WHEN salary >= 100000 THEN
        RETURN 'High';
    WHEN salary >= 50000 THEN
        RETURN 'Medium';
    ELSE
        RETURN 'Low';
END CASE;
```

This evaluates multiple Boolean conditions.

## 13.3 Example: Discount category

```
DELIMITER //

CREATE FUNCTION discount_category(
    amount DECIMAL(10,2)
)
RETURNS VARCHAR(20)
DETERMINISTIC
BEGIN
    CASE
        WHEN amount IS NULL OR amount < 0 THEN
            RETURN 'Invalid';
        WHEN amount >= 10000 THEN
            RETURN 'Premium';
        WHEN amount >= 5000 THEN
            RETURN 'Standard';
        ELSE
            RETURN 'Basic';
    END CASE;
END //

DELIMITER ;
```

Usage:

```
SELECT discount_category(12000);
SELECT discount_category(7000);
SELECT discount_category(2000);
```

Results:

```
Premium
Standard
Basic
```

## 13.4 `IF` versus `CASE`

Use `IF` when the logic is relatively simple and follows a small number of branches.

Use `CASE` when there are multiple categories or when a set of alternatives is easier to read in a structured form.

Both can implement equivalent logic in many situations.

---

# 14. Loops in MySQL Functions

Loops repeat a block of statements.

MySQL stored programs support constructs such as:

- `LOOP`
- `WHILE`
- `REPEAT`

Loops are useful when the number of iterations depends on a counter or condition.

However, for operations over table rows, set-based SQL is generally preferable to manually iterating through rows.

## 14.1 `WHILE` loop

### Syntax

```
WHILE condition DO
    statements;
END WHILE;
```

## Example: Sum numbers from 1 to n

```
DELIMITER //

CREATE FUNCTION sum_to_n(
    n INT
)
RETURNS BIGINT
DETERMINISTIC
BEGIN
    DECLARE i INT DEFAULT 1;
    DECLARE total BIGINT DEFAULT 0;

    IF n IS NULL OR n < 1 THEN
        RETURN 0;
    END IF;

    WHILE i <= n DO
        SET total = total + i;
        SET i = i + 1;
    END WHILE;

    RETURN total;
END //

DELIMITER ;
```

Usage:

```
SELECT sum_to_n(5);
```

Calculation:

```
1 + 2 + 3 + 4 + 5 = 15
```

Output:

```
15
```

## How the loop works

Initially:

```
i = 1
total = 0
```

During each iteration:

1. Add `i` to `total`.
2. Increase `i` by one.
3. Test whether `i <= n`.
4. Stop when the condition becomes false.

## 14.2 `REPEAT` loop

A `REPEAT` loop runs the statements and then tests the condition. Therefore, its body executes at least once.

Syntax:

```
REPEAT
    statements;
UNTIL condition
END REPEAT;
```

Example:

```
DELIMITER //

CREATE FUNCTION count_to_n(
    n INT
)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE counter INT DEFAULT 0;

    IF n IS NULL OR n < 1 THEN
        RETURN 0;
    END IF;

    REPEAT
        SET counter = counter + 1;
    UNTIL counter >= n
    END REPEAT;

    RETURN counter;
END //

DELIMITER ;
```

Usage:

```
SELECT count_to_n(5);
```

Output:

```
5
```

## 14.3 `LOOP` and `LEAVE`

A `LOOP` repeats indefinitely until a statement exits it.

Example:

```
DELIMITER //

CREATE FUNCTION count_to_n_using_loop(
    n INT
)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE counter INT DEFAULT 0;

    IF n IS NULL OR n < 1 THEN
        RETURN 0;
    END IF;

    count_loop: LOOP
        SET counter = counter + 1;

        IF counter >= n THEN
            LEAVE count_loop;
        END IF;
    END LOOP;

    RETURN counter;
END //

DELIMITER ;
```

Here:

- `count_loop` is a label.
- `LEAVE count_loop` exits the labeled loop.

Without a valid exit condition, a loop can run indefinitely or consume excessive resources.

### 14.4 When should you avoid loops?

For example, to calculate the total salaries of employees, use:

```
SELECT SUM(salary)
FROM employees;
```

Do not normally fetch every employee and add each salary manually in a stored function.

Set-based SQL is usually simpler and more efficient.

---

## 15. Functions with Mathematical Operations

Mathematical functions are among the simplest and most useful examples of stored functions.

### 15.1 Calculate the square

```
DELIMITER //

CREATE FUNCTION square_value(
    n DECIMAL(10,2)
)
RETURNS DECIMAL(20,4)
DETERMINISTIC
RETURN n * n;

//

DELIMITER ;
```

Usage:

```
SELECT square_value(5);
```

## 15.2 Calculate a percentage

```
DELIMITER //

CREATE FUNCTION calculate_percentage(
    part_value DECIMAL(10,2),
    total_value DECIMAL(10,2)
)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    IF part_value IS NULL
       OR total_value IS NULL
       OR total_value = 0 THEN
        RETURN NULL;
    END IF;

    RETURN (part_value / total_value) * 100;
END //

DELIMITER ;
```

Usage:

```
SELECT calculate_percentage(45, 60);
```

Output:

```
75.00
```

## 15.3 Calculate simple interest

Formula:

$$\text{Simple Interest} = \frac{P \times R \times T}{100}$$

Where:

- $P$ = principal amount.
- $R$ = annual interest rate in percent.
- $T$ = time in years.

```
DELIMITER //

CREATE FUNCTION simple_interest(
    principal DECIMAL(12,2),
    rate DECIMAL(5,2),
    years DECIMAL(5,2)
)
RETURNS DECIMAL(15,2)
DETERMINISTIC
BEGIN
    IF principal IS NULL OR rate IS NULL OR years IS NULL
       OR principal < 0 OR rate < 0 OR years < 0 THEN
        RETURN NULL;
    END IF;

    RETURN principal * rate * years / 100;
END //

DELIMITER ;
```

Usage:

```
SELECT simple_interest(10000, 5, 2);
```

Output:

```
1000.00
```

The function implements a simple-interest formula; actual financial applications may require explicit rounding and currency rules.

---

# 16. Functions with Strings

String functions are useful for cleaning, formatting, and transforming text.

## 16.1 Convert text to uppercase

```
DELIMITER //

CREATE FUNCTION make_upper(
    input_text VARCHAR(255)
)
RETURNS VARCHAR(255)
DETERMINISTIC
RETURN UPPER(input_text);

//

DELIMITER ;
```

Usage:

```
SELECT make_upper('mysql functions');
```

Output:

```
MYSQL FUNCTIONS
```

If the input is `NULL`, `UPPER()` returns `NULL`.

## 16.2 Generate a full name

```
DELIMITER //

CREATE FUNCTION full_name(
    first_name VARCHAR(100),
    last_name VARCHAR(100)
)
RETURNS VARCHAR(201)
DETERMINISTIC
RETURN CONCAT_WS(' ', first_name, last_name);

//

DELIMITER ;
```

Usage:

```
SELECT full_name('Asha', 'Patil');
```

Output:

```
Asha Patil
```

`CONCAT_WS()` skips `NULL` arguments after its separator, which is often useful for names.

## 16.3 Remove extra spaces

```
DELIMITER //

CREATE FUNCTION clean_text(
    input_text VARCHAR(255)
)
RETURNS VARCHAR(255)
DETERMINISTIC
RETURN TRIM(input_text);

//

DELIMITER ;
```

Usage:

```
SELECT clean_text('   Hello MySQL   ');
```

Output:

```
Hello MySQL
```

`TRIM()` removes leading and trailing spaces. It does not collapse every sequence of internal spaces.

## 16.4 Extract a substring

```
DELIMITER //

CREATE FUNCTION first_n_characters(
    input_text VARCHAR(255),
    character_count INT
)
RETURNS VARCHAR(255)
DETERMINISTIC
BEGIN
    IF input_text IS NULL
       OR character_count IS NULL
       OR character_count < 0 THEN
        RETURN NULL;
    END IF;

    RETURN LEFT(input_text, character_count);
END //

DELIMITER ;
```

Usage:

```
SELECT first_n_characters('Database', 4);
```

Output:

```
Data
```

## 16.5 Important string considerations

- MySQL string comparisons depend on the applicable collation.
- `LENGTH()` measures bytes, while `CHAR_LENGTH()` measures characters.
- `CONCAT()` returns `NULL` if an argument is `NULL`.
- `CONCAT_WS()` skips `NULL` values after the separator.
- Choose a suitable character set, collation, and return length for multilingual text.

---

# 17. Functions with Dates and Times

Date functions are commonly used in business applications.

## 17.1 Calculate the number of days between two dates

```
DELIMITER //

CREATE FUNCTION days_between(
    start_date DATE,
    end_date DATE
)
RETURNS INT
DETERMINISTIC
RETURN DATEDIFF(end_date, start_date);

//

DELIMITER ;
```

Usage:

```
SELECT days_between('2026-01-01', '2026-01-11');
```

Output:

```
10
```

`DATEDIFF()` returns the difference in calendar days, using the first date argument as the subtraction reference.

## 17.2 Calculate a future date

```
DELIMITER //

CREATE FUNCTION add_days(
    start_date DATE,
    number_of_days INT
)
RETURNS DATE
DETERMINISTIC
RETURN DATE_ADD(
    start_date,
    INTERVAL number_of_days DAY
);

//

DELIMITER ;
```

Usage:

```
SELECT add_days('2026-01-01', 30);
```

Output:

```
2026-01-31
```

## 17.3 Calculate a person's age

The following example calculates age in completed years.

```
DELIMITER //

CREATE FUNCTION calculate_age(
    birth_date DATE
)
RETURNS INT
NOT DETERMINISTIC
NO SQL
BEGIN
    IF birth_date IS NULL OR birth_date > CURDATE() THEN
        RETURN NULL;
    END IF;

    RETURN TIMESTAMPDIFF(YEAR, birth_date, CURDATE());
END //

DELIMITER ;
```

Usage:

```
SELECT calculate_age('2000-05-15');
```

The result depends on the current date.

This function is non-deterministic because the same birth date can produce different results on different days.

**Important:** Age calculations can have jurisdiction-specific requirements, particularly for leap-day birthdays. Use an explicitly defined policy if the result is legally or operationally significant.

### 17.4 Date-related built-in functions

| Function | Purpose |
|---|---|
| `CURDATE()` | Current date |
| `NOW()` | Current date and time |
| `DATEDIFF()` | Difference in days |
| `TIMESTAMPDIFF()` | Difference in a specified unit |
| `DATE_ADD()` | Add an interval |
| `DATE_SUB()` | Subtract an interval |
| `YEAR()` | Extract year |
| `MONTH()` | Extract month |
| `DAY()` | Extract day of month |

Use built-in date functions whenever they already meet the requirement. A stored function is useful when the calculation expresses a reusable business rule.

---

## 18. Functions That Read Table Data

A stored function can query tables and return one scalar value.

This is useful when the value depends on database records rather than only on input parameters.

### 18.1 Example database

```
CREATE TABLE products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100),
    price DECIMAL(10,2),
    stock_quantity INT
);
```

Insert sample records:

```
INSERT INTO products
VALUES
(1, 'Keyboard', 1500, 20),
(2, 'Mouse', 700, 50),
(3, 'Monitor', 12000, 10);
```

## 18.2 Return a product price

```
DELIMITER //

CREATE FUNCTION product_price(
    p_id INT
)
RETURNS DECIMAL(10,2)
READS SQL DATA
BEGIN
    DECLARE result_price DECIMAL(10,2) DEFAULT NULL;

    SELECT price
    INTO result_price
    FROM products
    WHERE product_id = p_id;

    RETURN result_price;
END //

DELIMITER ;
```

Usage:

```
SELECT product_price(2);
```

Output:

```
700.00
```

This function assumes that a matching product exists. If no product matches, the `SELECT ... INTO` statement raises a `NOT FOUND` condition.

## 18.3 Return the total number of products

```
DELIMITER //

CREATE FUNCTION total_products()
RETURNS BIGINT
READS SQL DATA
BEGIN
    DECLARE result_count BIGINT DEFAULT 0;

    SELECT COUNT(*)
    INTO result_count
    FROM products;

    RETURN result_count;
END //

DELIMITER ;
```

Usage:

```
SELECT total_products();
```

Output for the sample data:

```
3
```

## 18.4 Return a customer's total order amount

Suppose we have an orders table:

```
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    total_amount DECIMAL(12,2)
);
```

A function can calculate a customer's total:

```
DELIMITER //

CREATE FUNCTION customer_total(
    p_customer_id INT
)
RETURNS DECIMAL(15,2)
READS SQL DATA
BEGIN
    DECLARE result_total DECIMAL(15,2) DEFAULT 0;

    SELECT COALESCE(SUM(total_amount), 0)
    INTO result_total
    FROM orders
    WHERE customer_id = p_customer_id;

    RETURN result_total;
END //

DELIMITER ;
```

Usage:

```
SELECT customer_total(101);
```

The result is zero when the customer has no matching orders, assuming the query completes normally.

`SUM()` returns `NULL` if there are no non-`NULL` input values. `COALESCE()` converts that `NULL` to zero.

### 18.5 Why use `READS SQL DATA`?

The function reads table data but does not intentionally modify it.

This characteristic documents the function's data-access behavior.

It is an advisory declaration; MySQL does not use it as a complete enforcement mechanism for every statement inside the routine.

---

## 19. Working with `NULL` Values

`NULL` represents a missing or unknown value. It is not the same as zero or an empty string.

Understanding `NULL` is essential when designing reliable functions.

## 19.1 Example: Adding two values

```
DELIMITER //

CREATE FUNCTION safe_add(
    a DECIMAL(10,2),
    b DECIMAL(10,2)
)
RETURNS DECIMAL(11,2)
DETERMINISTIC
RETURN a + b;

//

DELIMITER ;
```

Usage:

```
SELECT safe_add(10, 20);
SELECT safe_add(10, NULL);
```

Results:

```
30.00
NULL
```

The arithmetic expression propagates `NULL`.

## 19.2 Using `COALESCE()`

If the business rule says that a missing value should be treated as zero:

```
DELIMITER //

CREATE FUNCTION add_null_as_zero(
    a DECIMAL(10,2),
    b DECIMAL(10,2)
)
RETURNS DECIMAL(11,2)
DETERMINISTIC
RETURN COALESCE(a, 0) + COALESCE(b, 0);

//

DELIMITER ;
```

Usage:

```
SELECT add_null_as_zero(10, NULL);
```

Output:

```
10.00
```

This is appropriate only when treating a missing input as zero is valid for the application.

### 19.3 Checking for `NULL`

Correct:

```
IF input_value IS NULL THEN
    RETURN NULL;
END IF;
```

Incorrect:

```
IF input_value = NULL THEN
    RETURN NULL;
END IF;
```

Use `IS NULL` and `IS NOT NULL` because ordinary equality comparisons with `NULL` produce an unknown result rather than `TRUE`.

### 19.4 Should a function return `NULL` or raise an error?

That depends on the business requirement.

Returning `NULL` may be appropriate when the result is unavailable or the input is intentionally optional.

An SQL error may be preferable when invalid input should stop a transaction or alert the caller.

For example, a financial calculation may need to reject an invalid principal instead of silently returning `NULL`.

---

## 20. Error Handling and Input Validation

A reliable function should define how it handles missing records, invalid input, and unexpected conditions.

MySQL supports condition handlers and `SIGNAL` for stored-program error handling.

## 20.1 `SIGNAL` an error for invalid input

Example:

```
DELIMITER //

CREATE FUNCTION checked_divide(
    numerator DECIMAL(12,4),
    denominator DECIMAL(12,4)
)
RETURNS DECIMAL(16,6)
DETERMINISTIC
BEGIN
    IF numerator IS NULL OR denominator IS NULL THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Inputs must not be NULL';
    END IF;

    IF denominator = 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Division by zero is not allowed';
    END IF;

    RETURN numerator / denominator;
END //

DELIMITER ;
```

Usage:

```
SELECT checked_divide(10, 2);
```

Output:

```
5.000000
```

But:

```
SELECT checked_divide(10, 0);
```

Raises a user-defined SQL exception.

`SQLSTATE '45000'` is commonly used for an application-defined exception.

## 20.2 Handling a missing row with `NOT FOUND`

Consider a function that looks up a product price.

A missing product can cause a `NOT FOUND` condition. You can handle that condition explicitly.

```
DELIMITER //

CREATE FUNCTION find_product_price(
    p_id INT
)
RETURNS DECIMAL(10,2)
READS SQL DATA
BEGIN
    DECLARE result_price DECIMAL(10,2) DEFAULT NULL;

    DECLARE CONTINUE HANDLER FOR NOT FOUND
        SET result_price = NULL;

    SELECT price
    INTO result_price
    FROM products
    WHERE product_id = p_id;

    RETURN result_price;
END //

DELIMITER ;
```

Now, when the query finds no row, the handler assigns `NULL`, and execution can continue to the `RETURN` statement.

This example assumes `product_id` is unique, as it is in the table definition.

## 20.3 Handler types

| Handler type | Meaning |
|---|---|
| `CONTINUE` | Handle the condition and continue with the next statement |
| `EXIT` | Handle the condition and exit the current compound block |
| `NOT FOUND` | Handles conditions such as a query returning no rows |
| `SQLEXCEPTION` | Handles SQL exceptions |

| Handler type | Meaning |
|---|---|
| `SQLWARNING` | Handles SQL warnings |

Handlers should be used carefully. A broad handler that suppresses every exception may hide real defects.

### 20.4 Best practices for error handling

- Validate parameters before performing calculations.
- Handle missing records deliberately.
- Do not silently convert every error to zero.
- Use clear error messages for invalid business inputs.
- Avoid swallowing exceptions without recording or communicating the problem.
- Test boundary values, `NULL`, zero, negative numbers, and missing rows.

---

## 21. Deterministic and Non-Deterministic Functions

Determinism is an important advanced concept in MySQL.

### 21.1 What does deterministic mean?

A function is deterministic when the same inputs always produce the same output under the function's defined behavior.

Example:

```
CREATE FUNCTION square_number(n INT)
RETURNS INT
DETERMINISTIC
RETURN n * n;
```

For a given input:

```
SELECT square_number(5);
```

The result is always `25`.

## 21.2 What is a non-deterministic function?

A function is non-deterministic when its output can change even if its explicit arguments remain the same.

Example:

```
DELIMITER //

CREATE FUNCTION current_timestamp_value()
RETURNS DATETIME
NOT DETERMINISTIC
NO SQL
RETURN NOW();

//

DELIMITER ;
```

The result changes over time.

Other sources of non-determinism include:

- Random-number generation.
- Current date and time.
- Reading data that can change between calls.
- Dependence on external or session state.

A function that reads a table is not automatically deterministic just because it receives the same arguments. The underlying table may change.

## 21.3 Why declare determinism correctly?

MySQL uses routine characteristics when evaluating certain operations, including behavior associated with binary logging and optimization.

Incorrectly declaring a function deterministic can lead to incorrect assumptions about its behavior.

For example:

```
DETERMINISTIC
```

should not be added merely to make a function definition easier to create.

Instead, describe the function accurately.

### 21.4 Comparison

| Function | Appropriate characteristic |
| --- | --- |
| Add two supplied integers | `DETERMINISTIC` |
| Calculate a square | `DETERMINISTIC` |
| Return `UPPER(input_text)` | Usually `DETERMINISTIC` |
| Return `NOW()` | `NOT DETERMINISTIC` |
| Return a random number | `NOT DETERMINISTIC` |
| Return the current stored price | Depends on mutable table data; do not assume deterministic behavior |

---

## 22. SQL Data Access Characteristics

MySQL stored functions support several data-access characteristics.

### 22.1 `NO SQL`

Declares that the function contains no SQL statements.

Example:

```
DELIMITER //

CREATE FUNCTION square_no_sql(
    n INT
)
RETURNS INT
DETERMINISTIC
NO SQL
RETURN n * n;

//

DELIMITER ;
```

### 22.2 `CONTAINS SQL`

Indicates that the function contains SQL statements but does not read or write data.

This is the default characteristic when none of the data-access characteristics is specified.

### 22.3 `READS SQL DATA`

Indicates that the function reads data.

Example:

```
READS SQL DATA
```

A function that selects a product price from a table is a typical example.

### 22.4 `MODIFIES SQL DATA`

Indicates that the function may modify data.

However, this characteristic does not override the restrictions imposed on stored functions.

MySQL stored functions have significant limitations on statements that change data. In particular, a function generally cannot modify a table that is already being used by the statement that invoked it, and functions are subject to restrictions involving transaction control and data modification.

Do not use `MODIFIES SQL DATA` as permission to build unrestricted data-writing logic into a function.

## 22.5 Important clarification

These characteristics are advisory descriptions. MySQL does not treat them as complete enforcement rules that make an otherwise prohibited statement permissible.

Choose the characteristic that truthfully describes the routine.

---

# 23. Security and Permissions

Stored functions execute inside the database server, so their privileges and execution context matter.

## 23.1 Creating a function

Creating a stored function generally requires the `CREATE ROUTINE` privilege.

Depending on the server configuration, binary logging and routine-creation policies can impose additional requirements.

## 23.2 Executing a function

The caller generally needs the appropriate `EXECUTE` privilege.

The privileges used to access tables inside the function depend on the function's security context.

### 23.3 `SQL SECURITY DEFINER`

This is the default security context.

The function executes using the privileges of its definer, subject to MySQL's stored-object security rules.

Example:

```
CREATE FUNCTION example_function()
RETURNS INT
DETERMINISTIC
SQL SECURITY DEFINER
RETURN 1;
```

The function's definer must have the privileges required for its operations.

### 23.4 `SQL SECURITY INVOKER`

The function executes using the invoking user's privileges.

Example:

```
CREATE FUNCTION example_function()
RETURNS INT
DETERMINISTIC
SQL SECURITY INVOKER
RETURN 1;
```

This can be useful when access to underlying objects should be governed by the caller's permissions.

## 23.5 Security best practices

- Use least-privilege accounts.
- Review the function's definer.

- Choose `DEFINER` or `INVOKER` deliberately.
- Avoid exposing sensitive table data through functions.
- Avoid dynamically constructing SQL from untrusted inputs.
- Review the impact of routine changes on all callers.
- Understand binary logging and replication requirements before deployment.

---

# 24. Restrictions of Stored Functions

Stored functions have restrictions that distinguish them from procedures.

These restrictions are important for production development.

## 24.1 No direct result-set return

A stored function cannot return a result set to the client.

This is not valid as a way to return a result set from a function:

```
SELECT *
FROM products;
```

Instead, retrieve a single value:

```
SELECT price
INTO result_price
FROM products
WHERE product_id = p_id;
```

Then return that value:

```
RETURN result_price;
```

## 24.2 No explicit transaction control

A stored function cannot perform transaction-control statements such as:

```
COMMIT;
```

or:

```
ROLLBACK;
```

These are not allowed inside stored functions.

## 24.3 Recursion is not supported

MySQL stored functions cannot be recursive.

If a calculation requires repeated steps, consider a loop, a procedure, a recursive query where appropriate, or application-side logic.

## 24.4 Restrictions on table modifications

Stored functions have restrictions on data-modifying statements.

In particular, they cannot modify a table that is already being used by the statement that invoked the function. This can lead to errors when a function attempts to update the same table being queried or modified by the outer statement.

A procedure or application workflow is usually more appropriate for complex data modifications.

## 24.5 No `USE` statement inside a stored routine

A stored routine is associated with a database. MySQL establishes that database context when the routine executes.

You cannot change the current database using `USE` inside the routine.

## 24.6 Privileges and binary logging

Stored-function creation can be subject to additional restrictions when binary logging is enabled.

For example, `log_bin_trust_function_creators` can affect whether additional checks are required.

The exact requirements depend on server version, configuration, and privileges.

## 24.7 Other considerations

- Avoid using names that conflict with built-in functions.
- Be careful with stored SQL modes and character sets.
- Do not assume that all clients support the same delimiter syntax.
- Test functions under the actual MySQL version and configuration used in production.

Official reference: https://dev.mysql.com/doc/refman/8.4/en/create-procedure.html

---

# 25. Advanced Examples and Real-World Use Cases

The following examples combine the concepts introduced earlier.

## 25.1 Product discount calculator

Business rule: subtract a percentage discount from the original price.

```
DELIMITER //

CREATE FUNCTION discounted_price(
    original_price DECIMAL(12,2),
    discount_percent DECIMAL(5,2)
)
RETURNS DECIMAL(12,2)
DETERMINISTIC
BEGIN
    IF original_price IS NULL
       OR discount_percent IS NULL
       OR original_price < 0
       OR discount_percent < 0
       OR discount_percent > 100 THEN
        RETURN NULL;
    END IF;

    RETURN ROUND(
        original_price * (1 - discount_percent / 100),
        2
    );
END //

DELIMITER ;
```

Usage:

```
SELECT discounted_price(2000, 15);
```

Output:

```
1700.00
```

Explanation:

- Original price: 2000.
- Discount: 15%.
- Discount amount: 300.

- Final price: 1700.

This example uses `NULL` to indicate invalid input. A production application might instead raise an exception.

## 25.2 Salary increment calculator

Business rule: calculate the new salary after a percentage increment.

```
DELIMITER //

CREATE FUNCTION new_salary(
    current_salary DECIMAL(12,2),
    increment_percent DECIMAL(5,2)
)
RETURNS DECIMAL(12,2)
DETERMINISTIC
BEGIN
    IF current_salary IS NULL
       OR increment_percent IS NULL
       OR current_salary < 0
       OR increment_percent < 0 THEN
        RETURN NULL;
    END IF;

    RETURN ROUND(
        current_salary * (1 + increment_percent / 100),
        2
    );
END //

DELIMITER ;
```

Usage:

```
SELECT new_salary(50000, 10);
```

Output:

```
55000.00
```

## 25.3 Employee annual salary

```
DELIMITER //

CREATE FUNCTION annual_salary(
    monthly_salary DECIMAL(12,2)
)
RETURNS DECIMAL(14,2)
DETERMINISTIC
BEGIN
    IF monthly_salary IS NULL OR monthly_salary < 0 THEN
        RETURN NULL;
    END IF;

    RETURN monthly_salary * 12;
END //

DELIMITER ;
```

Usage:

```
SELECT annual_salary(40000);
```

Output:

```
480000.00
```

This represents twelve months of base salary and does not include bonuses, deductions, or other compensation.

## 25.4 Stock availability checker

Suppose the `products` table contains a `stock_quantity` column.

```
DELIMITER //

CREATE FUNCTION is_product_available(
    p_id INT
)
RETURNS TINYINT
READS SQL DATA
BEGIN
    DECLARE quantity_available INT DEFAULT NULL;

    DECLARE CONTINUE HANDLER FOR NOT FOUND
        SET quantity_available = NULL;

    SELECT stock_quantity
    INTO quantity_available
    FROM products
    WHERE product_id = p_id;

    IF quantity_available IS NULL THEN
        RETURN NULL;
    ELSEIF quantity_available > 0 THEN
        RETURN 1;
    ELSE
        RETURN 0;
    END IF;
END //

DELIMITER ;
```

Usage:

```
SELECT is_product_available(1);
```

Interpretation:

- `1`: product exists and is in stock.
- `0`: product exists but is out of stock.
- `NULL`: product does not exist or the stored quantity is `NULL`.

This example distinguishes unavailable information from a definite out-of-stock result.

## 25.5 Customer spending classification

```
DELIMITER //

CREATE FUNCTION spending_level(
    total_spent DECIMAL(15,2)
)
RETURNS VARCHAR(20)
DETERMINISTIC
BEGIN
    IF total_spent IS NULL OR total_spent < 0 THEN
        RETURN 'Invalid';
    ELSEIF total_spent >= 100000 THEN
        RETURN 'Platinum';
    ELSEIF total_spent >= 50000 THEN
        RETURN 'Gold';
    ELSEIF total_spent >= 10000 THEN
        RETURN 'Silver';
    ELSE
        RETURN 'Standard';
    END IF;
END //

DELIMITER ;
```

Usage:

```
SELECT
    spending_level(150000),
    spending_level(75000),
    spending_level(20000),
    spending_level(3000);
```

Output:

```
Platinum | Gold | Silver | Standard
```

## 25.6 Combine a function with a table query

Assuming the `employees` table from earlier exists:

```
SELECT
    employee_id,
    employee_name,
    salary,
    new_salary(salary, 10) AS salary_after_increment
FROM employees;
```

This is a common pattern: the function calculates one result for each employee row.

For large tables, consider whether the formula can be expressed directly in SQL, because invoking a stored function per row may add overhead.

---

# 26. Function Management Commands

After creating functions, you need to know how to inspect, replace, and remove them.

## 26.1 Show stored functions

```
SHOW FUNCTION STATUS;
```

To filter by database:

```
SHOW FUNCTION STATUS
WHERE Db = 'your_database_name';
```

Replace the database name with the actual schema name.

## 26.2 Display a function's definition

```
SHOW CREATE FUNCTION add_numbers;
```

This displays the function definition and relevant metadata.

## 26.3 Query `INFORMATION_SCHEMA`

```
SELECT
    ROUTINE_SCHEMA,
    ROUTINE_NAME,
    ROUTINE_TYPE,
    DATA_TYPE
FROM INFORMATION_SCHEMA.ROUTINES
WHERE ROUTINE_TYPE = 'FUNCTION';
```

This can be used to inspect stored functions visible to the current account.

### 26.4 Delete a function

```
DROP FUNCTION add_numbers;
```

This removes the stored function.

A safer form when the function may not exist is:

```
DROP FUNCTION IF EXISTS add_numbers;
```

Be careful: dropping a function breaks applications and queries that depend on it.

### 26.5 Modify an existing function

MySQL does not provide a general `ALTER FUNCTION` operation for replacing the body of an existing stored function.

The common approach is to drop and recreate the function, or use a deployment process that manages the replacement safely.

For example:

```
DROP FUNCTION IF EXISTS add_numbers;
```

Then execute the new `CREATE FUNCTION` definition.

Plan the migration carefully to avoid leaving the application without the function during deployment.

---

## 27. Debugging and Common Errors

### 27.1 Error: function already exists

Cause: a function with that name already exists in the schema.

Solution:

```
DROP FUNCTION IF EXISTS add_numbers;
```

Then recreate it.

### 27.2 Error: incorrect delimiter

Cause: the SQL client interpreted an internal semicolon as the end of the definition.

Solution: in clients that support it, set a temporary delimiter before creating the function.

```
DELIMITER //
```

Restore it afterward:

```
DELIMITER ;
```

## 27.3 Error: function does not exist

Possible causes:

- The function was not created successfully.
- You are using the wrong database.
- The name is incorrect.
- The current account cannot execute it.

Check:

```
SELECT DATABASE();
```

And:

```
SHOW FUNCTION STATUS;
```

You can also call the function with its schema name:

```
SELECT your_database_name.add_numbers(10, 20);
```

## 27.4 Error: `SELECT ... INTO` returns multiple rows

Cause: the query matches more than one row.

Solution: use a unique lookup condition, such as a primary key, when the function is intended to retrieve a single value.

Do not arbitrarily use `LIMIT 1` unless selecting one matching row is actually the intended business rule.

## 27.5 Error: no data found

Cause: `SELECT ... INTO` did not find a matching row.

Solution: use an appropriate `NOT FOUND` handler or define a clear missing-record policy.

### 27.6 Error: access denied

Cause: the account lacks the necessary privileges or the server's routine-creation policies impose additional requirements.

Solution: check the account's permissions and the server's binary-logging configuration. Ask a database administrator to grant only the required privileges.

### 27.7 Error: function returns the wrong value

Possible causes:

- Incorrect condition ordering.
- Wrong formula.
- `NULL` propagation.
- Insufficient numeric precision.
- Unexpected input conversion.
- A table lookup returning different data than expected.

Test individual inputs and edge cases before deploying the function.

---

## 28. Performance Optimization

Functions can improve maintainability, but they should be designed with performance in mind.

### 28.1 Avoid unnecessary function calls

Suppose a table has one million rows.

```
SELECT
    employee_id,
    square_number(salary)
FROM employees;
```

The function may be evaluated for a large number of rows.

If the logic is a simple expression, compare it with:

```
SELECT
    employee_id,
    salary * salary
FROM employees;
```

The direct expression may be simpler and may avoid stored-function call overhead.

## 28.2 Avoid repeated table lookups

Consider a function that performs a `SELECT` for each product row.

```
SELECT
    product_id,
    product_price(product_id)
FROM products;
```

If the function queries `products` again for every row, the query performs unnecessary lookups.

Prefer selecting the required columns directly:

```
SELECT
    product_id,
    price
FROM products;
```

This is particularly important for large datasets.

## 28.3 Index lookup columns

If a function looks up a row using:

```
WHERE product_id = p_id
```

make sure `product_id` has an appropriate index.

A primary key or unique index is often suitable.

## 28.4 Do not use functions unnecessarily in filtering conditions

For example:

```
SELECT *
FROM employees
WHERE YEAR(hire_date) = 2025;
```

Applying a function to an indexed column may prevent a straightforward index range lookup in some circumstances.

A range condition is often preferable:

```
SELECT *
FROM employees
WHERE hire_date >= '2025-01-01'
  AND hire_date < '2026-01-01';
```

This can make better use of an index on `hire_date`.

The exact execution plan depends on the schema, indexes, MySQL version, and data distribution. Verify performance using `EXPLAIN` and representative data.

### 28.5 Use the correct numeric type

Use `DECIMAL` for exact monetary arithmetic when appropriate.

Avoid unnecessarily narrow return types that could truncate or overflow expected values.

### 28.6 Prefer set-based SQL for bulk operations

Instead of iterating through rows and processing each one individually, use SQL operations such as:

```
SUM()
```

```
COUNT()
```

```
CASE
```

```
UPDATE ... WHERE ...
```

The best design minimizes unnecessary row-by-row processing.

---

## 29. Best Practices

Use this checklist when writing production-ready stored functions.

### Design

- Give functions descriptive names.
- Make each function responsible for one clear task.
- Use functions when the logic is genuinely reusable.
- Avoid making simple SQL unnecessarily complicated.

## Parameters and return values

- Choose appropriate parameter and return types.
- Use sufficient numeric precision.
- Define what happens with invalid inputs.
- Decide whether missing information should produce `NULL` or an error.
- Document units, such as percentages versus decimal rates.

## SQL logic

- Use `IF` or `CASE` for readable conditions.
- Use local variables only when they improve clarity.
- Avoid loops when a set-based SQL expression is simpler.
- Use unique indexes for single-row lookups.
- Handle missing rows explicitly.

## Reliability

- Declare determinism accurately.
- Describe SQL data access accurately.
- Handle expected exceptions.
- Avoid suppressing unexpected errors.
- Test zero, negative, boundary, and `NULL` values.

## Security

- Use least-privilege accounts.
- Review the definer and security context.
- Avoid exposing sensitive data.
- Follow the server's binary-logging and deployment requirements.

## Performance

- Avoid repeated table lookups.
- Check execution plans.
- Benchmark functions on realistic datasets.
- Prefer direct expressions when they are clearer and more efficient.

---

# 30. Interview Questions and Answers

## Q1. What is a user-defined function in MySQL?

A stored function is a reusable SQL routine created with `CREATE FUNCTION`. It accepts input parameters and returns one scalar value.

## Q2. What is the difference between a stored function and a stored procedure?

A function returns a scalar value and can be used in SQL expressions. A procedure is invoked with `CALL` and can perform multi-step operations and return result sets or output parameters.

## Q3. Is `RETURN` mandatory in a stored function?

A stored function must return a value. Its definition must provide a valid execution path that returns the declared result type.

## Q4. Can a function have multiple parameters?

Yes.

```
CREATE FUNCTION add_numbers(a INT, b INT)
RETURNS INT
DETERMINISTIC
RETURN a + b;
```

## Q5. Can a function have no parameters?

Yes. Use an empty parameter list:

```
CREATE FUNCTION get_value()
RETURNS INT
DETERMINISTIC
RETURN 100;
```

## Q6. Why do we use `DELIMITER`?

In command-line clients that support it, `DELIMITER` changes the client-side statement terminator so that semicolons inside a stored routine do not prematurely terminate the full definition.

## Q7. What is a deterministic function?

A deterministic function returns the same result for the same inputs under its defined behavior.

## Q8. What is a non-deterministic function?

A non-deterministic function may return different results for the same explicit inputs because of factors such as current time or random values.

## Q9. Can a stored function return multiple rows?

No. A stored function returns one scalar value. A stored procedure can return a result set.

## Q10. Can a function use `SELECT`?

Yes, but it cannot use a `SELECT` that returns a result set to the caller. It can use `SELECT ... INTO` to retrieve scalar values into local variables.

## Q11. Can a stored function modify a table?

Stored functions are subject to restrictions on data modification. In particular, they cannot modify a table already being used by the statement that invoked the function. Procedures are generally a better choice for data-changing workflows.

## Q12. Can a function contain `COMMIT` or `ROLLBACK`?

No. Explicit transaction-control statements such as `COMMIT` and `ROLLBACK` are not permitted inside stored functions.

## Q13. Can stored functions be recursive in MySQL?

No. MySQL stored functions cannot be recursive.

## Q14. What is `READS SQL DATA`?

It declares that the routine reads SQL data. This is a descriptive characteristic rather than a complete enforcement mechanism.

## Q15. What is `NO SQL`?

It declares that the function contains no SQL statements.

## Q16. What is `SQL SECURITY DEFINER`?

It specifies that the function executes using the privileges of its definer, subject to MySQL's stored-object security rules.

## Q17. What is `SQL SECURITY INVOKER`?

It specifies that the function executes using the invoking user's privileges.

## Q18. What is the purpose of `DECLARE`?

It declares local variables and other permitted objects used within a stored-program block.

## Q19. What is the difference between `SET` and `SELECT ... INTO`?

`SET` assigns an expression to a variable. `SELECT ... INTO` assigns query results to variables.

## Q20. How do you handle a missing row?

Use a `NOT FOUND` handler or explicitly define a missing-record policy.

## Q21. How do you remove a function?

```
DROP FUNCTION IF EXISTS function_name;
```

## Q22. How can you view a function's definition?

```
SHOW CREATE FUNCTION function_name;
```

## Q23. Why might functions slow down queries?

Stored-function invocation overhead and repeated internal queries can become expensive when a function is evaluated for many rows.

## Q24. When should you use a procedure instead?

Use a procedure when you need result sets, output parameters, transaction management, or a multi-step workflow.

### Q25. What is the difference between a stored function and a loadable UDF?

A stored function is written in SQL and stored in the database. A loadable UDF is implemented in compiled code or provided through a supported server extension.

---

# 31. Practice Exercises

Try solving these exercises independently before checking your solutions.

## Beginner level

1. Create a function that adds two integers.
2. Create a function that returns the square of a number.
3. Create a function that calculates the cube of a number.
4. Create a function that returns whether a number is positive, negative, or zero.
5. Create a function that converts a string to uppercase.
6. Create a function that joins a first name and last name.
7. Create a function that calculates simple interest.
8. Create a function that calculates the percentage of a total.

## Intermediate level

9. Create a function that assigns grades to marks.
10. Create a function that calculates a salary after a 10% increment.
11. Create a function that calculates a discounted price.
12. Create a function that calculates the number of days between two dates.
13. Create a function that returns the sum of integers from 1 to `n`.
14. Create a function that retrieves the price of a product by ID.
15. Create a function that returns the total order amount for a customer.
16. Create a function that handles a missing row using a condition handler.

## Advanced level

17. Create a function that classifies customers according to total spending.
18. Create a function that validates the input to a division operation and raises an error when the denominator is zero.

19. Create a function that checks product availability.
20. Compare a row-by-row function lookup with an equivalent join or direct query.
21. Test how `NULL` inputs affect different mathematical and string functions.
22. Investigate the behavior of a function when its SQL security context changes.
23. Use `EXPLAIN` to compare queries with direct expressions and stored-function calls.
24. Design a deployment migration that replaces a stored function without leaving dependent applications in an inconsistent state.

---

# 32. Quick Revision Cheat Sheet

## Create a function

```
CREATE FUNCTION function_name(parameter_name INT)
RETURNS INT
DETERMINISTIC
RETURN parameter_name * 2;
```

## Call a function

```
SELECT function_name(10);
```

## Declare a local variable

```
DECLARE total INT DEFAULT 0;
```

## Assign a value

```
SET total = 100;
```

## Read one value from a table

```
SELECT price
INTO result_price
FROM products
WHERE product_id = p_id;
```

## Use a condition

```
IF condition THEN
    RETURN value_1;
ELSE
    RETURN value_2;
END IF;
```

## Handle a missing row

```
DECLARE CONTINUE HANDLER FOR NOT FOUND
    SET result_value = NULL;
```

## Raise an error

```
SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Invalid input';
```

## Drop a function

```
DROP FUNCTION IF EXISTS function_name;
```

## View a function definition

```
SHOW CREATE FUNCTION function_name;
```

## Core concepts to remember

- `CREATE FUNCTION` defines a stored function.
- `RETURNS` specifies the result type.
- `RETURN` returns the scalar result.
- `DECLARE` creates local variables.
- `SET` assigns values.
- `SELECT ... INTO` retrieves scalar query results.
- `IF` and `CASE` implement conditional logic.
- `WHILE`, `REPEAT`, and `LOOP` implement repetition.
- `DETERMINISTIC` describes repeatable results for the same inputs.

- `READS SQL DATA` describes data-reading behavior.
- `SQL SECURITY DEFINER` and `INVOKER` determine the privilege context.
- Stored functions cannot return result sets or perform explicit transaction control.
- Functions should be optimized carefully when called for large numbers of rows.

---

# 33. Official MySQL References

Use the official MySQL documentation to verify details for your server version.

1. **CREATE PROCEDURE and CREATE FUNCTION syntax**
   https://dev.mysql.com/doc/refman/8.4/en/create-procedure.html
2. **Stored routine syntax and concepts**
   https://dev.mysql.com/doc/refman/8.4/en/stored-routines-syntax.html
3. **Restrictions on stored programs**
   https://dev.mysql.com/doc/refman/8.4/en/stored-program-restrictions.html
4. **MySQL server loadable functions**
   https://dev.mysql.com/doc/refman/8.4/en/server-loadable-functions.html
5. **Stored objects and access control**
   https://dev.mysql.com/doc/refman/8.4/en/stored-objects.html

---

# Final Summary

MySQL stored functions allow developers to define reusable database logic that accepts input parameters and returns a single scalar value.

A strong understanding of functions requires more than knowing the `CREATE FUNCTION` syntax. You should also understand parameters, return types, local variables, control flow, table lookups, `NULL` behavior, error handling, determinism, security, restrictions, and performance.

The best learning sequence is:

1. Create simple mathematical functions.
2. Practise parameters and return types.
3. Learn variables and conditional statements.
4. Use loops where appropriate.
5. Build functions that read table data.

6. Add validation and error handling.
7. Study security and function restrictions.
8. Compare performance with direct SQL queries.
9. Apply the concepts to realistic business problems.

**The most important principle:** Use a stored function when it makes a reusable scalar calculation clearer and easier to maintain. Use ordinary SQL expressions, joins, aggregates, or stored procedures when they are better suited to the task.